Concurrent Programming: Languages and techniques

Rust lifetimes

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Computer Science,
NOVA School of Science and Technology

Rust Lifetimes

Scope of a reference

Lifetimes – how long is a reference "alive"

- Every reference lives during a well-defined period of time, within a program scope
- Lifetime of references usually implicit (*inferred* by the type system)
- When not unique, lifetimes need to be declared by the programmer (*i. e.*, requires parameter annotations)

Infamous compile-time error

```
error[E0597]: `x` does not live long enough
```

(for some reference `x`)

What do you mean, ‘lifetime’?

```rust
fn main() {
    let r;
    {
        let x = 5;
        r = &x;
    }
    println!("r: {}", r);
}
```

`Variable ‘x‘ does not live long enough:`

Reference `r` points to it and lives longer

What do you mean, ‘lifetime’?

```
fn main() {
    let r;                      // ----------+-- 'a
                                //           |
    {                           //           |
        let x = 5;              // -+-- 'b   |
        r = &x;                 //  |        |
    }                           // -+        |
                                //           |
    println!("r: {}", r);       //           |
}                               // ----------+
```

`Variable ‘x‘ does not live long enough:`

Rust compiler determines that `r` has larger lifetime (`’a`) than `x` (which has lifetime `’b`)

What do you mean, ‘lifetime’?

```
fn main() {
    let x = 5;                  // -----------+-- 'b
                                //            |
    let r = &x;                 // --+-- 'a   |
                                //   |        |
    println!("r: {}", r);       //   |        |
                                // --+        |
}
```

All good now

Lifetime of `x` is longer than that of `r`

What do you mean, ‘lifetime’?

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

Both parameters must live *at least as long as* the returned value

What do you mean, 'lifetime'?

```rust
fn main() {
    let string1 = String::from("long string is long");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
    }
        println!("The longest string is {}", result);
}
```

Error:
`string2` does not live as long as the outer scope,
where `string1` and `scope` live.

What do you mean, 'lifetime'?

```rust
fn main() {
    let string1 = String::from("long string is long");

    {
        let string2 = String::from("xyz");
        let result = longest(string1.as_str(), string2.as_str()),
        println!("The longest string is {}", result);
    }
}
```

All good now:
`string1` lives as long as the outer scope,

and `string1` and `scope` live inside it (is at least as long!).

Subtyping lifetimes

We can declare lifetime `'b` as a subtype of `'a`
(to ensure that `'b` *lives at least as long as* `'a`)

```
fn get_int_ref3<'a,'b:'a>(x:  &'a i32, y:  &'b i32) ->
&'a i32 {
  if x > y {
    x
  } else {
    y
  }
}
```

Another example

However...

```rust
struct UserRef {
    active: bool,
    username: &str,
    email: &str,
    sign_in_count: u64,
}
```

```rust
fn main() {
    let user1 = UserRef {
        active: true,
        username: "someusername123",
        email: "someone@example.com",
        sign_in_count: 1,
    };
}
```

```
error[E0106]: missing lifetime specifier
 --> src/main.rs:3:15
  |
3 |     username: &str,
  |               ^ expected named lifetime paramet
  |
help: consider introducing a named lifetime paramet
  |
1 ~ struct User<'a> {
2 |     active: bool,
3 ~     username: &'a str,
  |
```

How can the compiler know how long should refs live?

"How long" is not actually time, but *space*

- A lifetime is the *name of a region*
- it identifies a block/scope of code somewhere in a program
- A reference tagged with a lifetime has to be valid for that entire region

Garbage collected languages hide lifetimes because the GC ensures that everything "magically" lives as long as it needs to.

Role of a lifetime

***Identify* where memory cannot be freed**

- to avoid holding a reference to something out of scope or meanwhile mutated
- it says how long is a reference valid (allows to track borrows across functions)

Rust allows to hide lifetime declarations, but actually all references have them:

fn `foo(&A) -> &B`; is sugar for
fn `foo<'a>(&'a A) -> &'a B`;

fn `foo(&A,&B,&C)`; is sugar for
fn `foo<'a,'b,'c>(&'a A,&'b B,&'c C)`;

fn `foo(&self,&B,&C) -> &D`; is sugar for
fn `foo<'a,'b,'c>(&'a self,&'b B,&'c C) -> &'a D`;

What does fn `foo<'a>(&'a A) -> &'a B` mean?

The input must live at least as long as the output

- if the output is used a lot (or for a long time), the region the input must be valid for is expanded
- once the output is no longer used, the compiler knows the input can be freed

With this system, Rust ensures:

- nothing is used after free
- nothing is mutated while outstanding references exist

How does the compiler figure out the lifetime of each reference?

It solves a constraint system:

takes all pairs (reference,lifetime) and tries to assign to every reference have a (minimal) concrete region

if it succeeds, the program compiles;
otherwise, one gets an error (something didn't live long enough)

ATTENTION:

lifetimes are only associated with references

if one uses them with "normal" variables, one gets a compile error.

More examples

From the tutorials:

On lists:

`https://rust-unofficial.github.io/too-many-lists/`

On lifetimes:

`https://www.youtube.com/watch?v=1QoT9fmPYr8`

Note:

id of a lifetime parameter is not important

usually, the fst is a, the snd b, etc

More examples – I

```
const FORTY_TWO: i32 = 42;
// lifetime is the full program:  'static

fn main() {
  let n = FORTY_TWO;
  let aref = get_int_ref(&n);
  println!("{}",aref)
}

fn get_int_ref<'lft>(x:  &'lft i32) -> &'lft i32 {
  x
}
```

More examples – II

If a reference is not returned from a function, no lifetime annotation is needed:

```
fn main() {
  let n = FORTY_TWO;
  let m = 42;
  let aref = get_int_ref2(&n,&m);
  println!("{}",aref)
}

fn get_int_ref2<'lft>(x:  &'lft i32, y:  &i32) ->
&'lft i32 {
  println!("{}",y);
  // mem of m no longer needed -> no lifetime needed
  x
}
```

Further issues

Static lifetime (the whole program)

only applies to constants or static variables

fn `get_int_ref<'static>(x: &'static i32, y: &'static i32) -> &'static i32`

Only OK if called with constant arguments; if one mixes const args with vars, use the more general version:

fn `get_int_ref<'a>(x: &'a i32, y: &'a i32) -> &'a i32`

What about using lifetimes with generics?

fn `get_ref<'a,T>(x: &'a T, y: &'a T) -> &'a T`